

Programmation mobile avec Flutter

SIL2 & RIT2 – OZA - FREV2026

PLAN

- 
- Introduction à Flutter
 - Les bases de Dart
 - Introduction aux widgets
 - Exemples de widgets
 - Les routes
 - Consommation des web services
 - Firebase Cloud Firestore



Introduction à Flutter

Qu'est-ce que Flutter ?

- Flutter est un Framework d'interface utilisateur open source de Google.
- Flutter permettant de créer des applications de haute qualité sur iOS, Android, Web, Bureau et Embedded à partir d'une seule base de code.
- Flutter permettant de créer des applications multiplateformes compiler en natif.
- La première version de Flutter a été publié le 4 décembre 2018.
- Dans la version 2.0.0 de Flutter plusieurs bugs et problème ont été résolus, de plus la version Web est désormais qualifiée de stable. Cette version a été publié le 6 mai 2021.

Qu'est-ce que Dart ?

- Dart est un langage de programmation pour les applications multiplateformes.
- Développé par Google.
- Dart est un langage orienté objet.
- Dart se compile en code natif.

The image features a dark blue background. On the left side, there are two vertical, wavy white lines that create a sense of movement or a stylized border. The word "Technologies" is written in a large, white, sans-serif font, centered horizontally on the right side of the image.

Technologies

Applications natives

Android

- Java
- Kotlin (2017)

iOS

- Objective-C
- Swift (2014)

Avantages

- Performances
- Interface spécifique à chaque plateforme

Inconvénients

- 1 application par plateforme
- Coûts et temps de développement

Applications hybrides

- Cordova, Ionic, ...
- Basées sur des technologies web (HTML, JS, CSS, ...)
- Utilisent les WebView (navigateur web intégré)

Avantages

- Code partagé
- Réutilisation

Inconvénients

- Performances
- Sécurité

Applications cross-plateformes

- Exemples de framework : React Native, Xamarin, Flutter
 - Objectif : générer 1 application Android et iOS à partir du même code
-

Avantages

- Code partagé / réutilisation
- Performances

Inconvénients

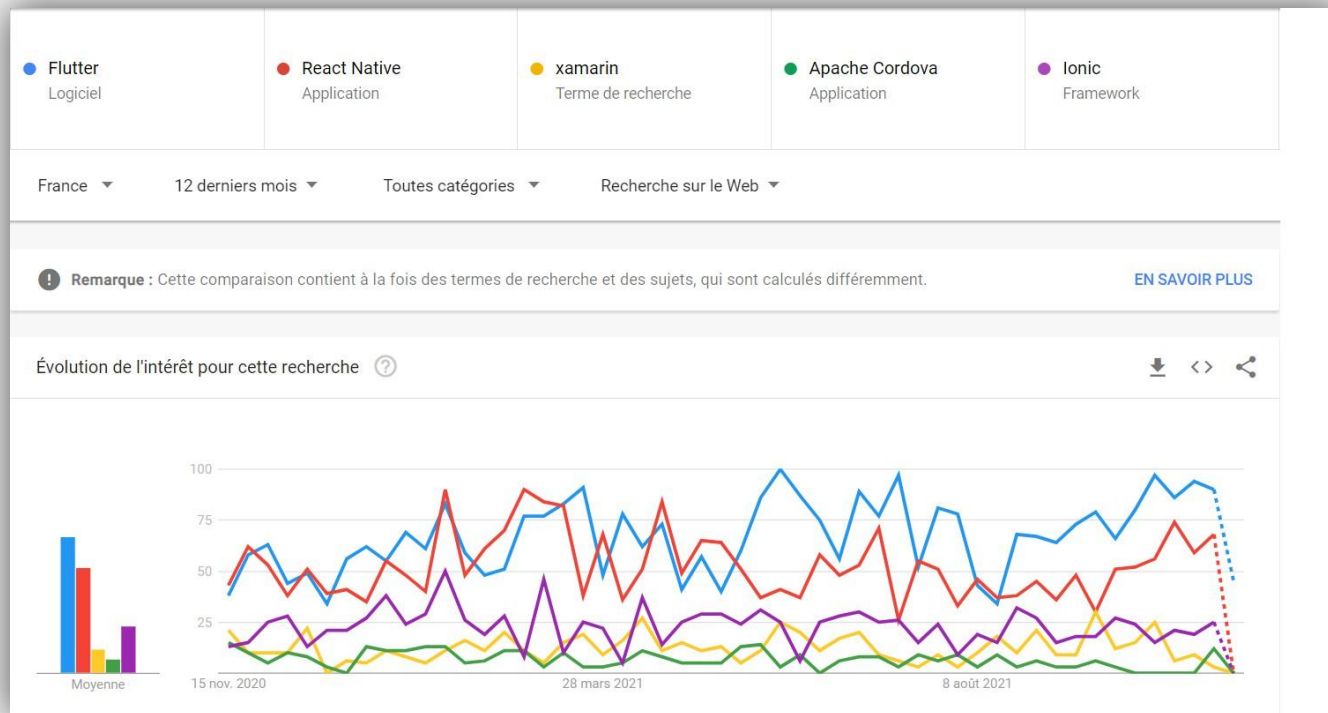
- Peut nécessiter d'écrire du code spécifique

Quelques chiffres



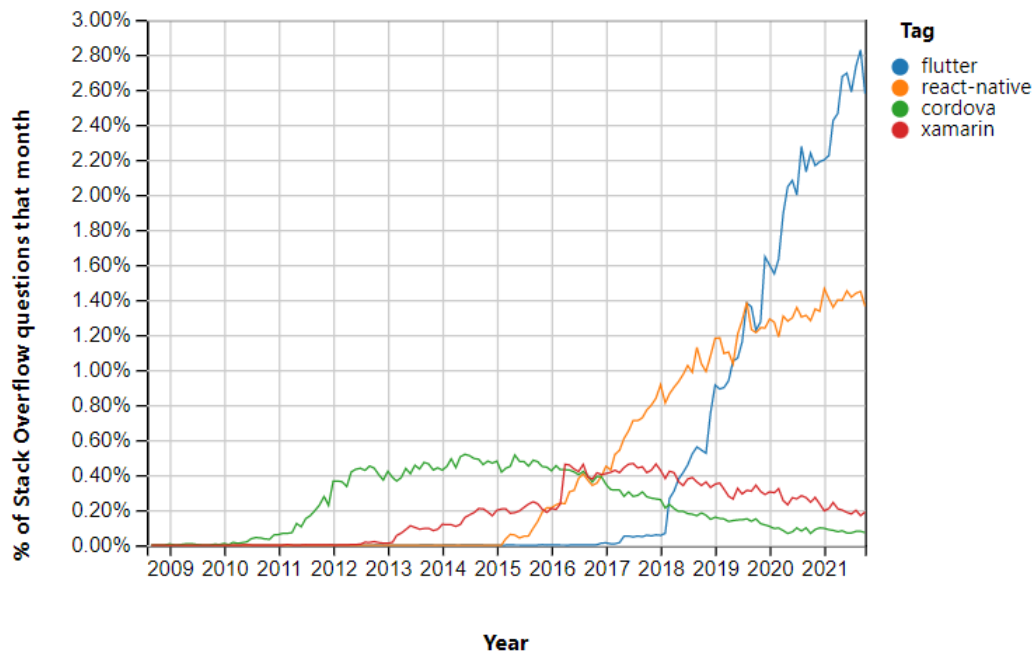
Google Trends

QUELQUES CHIFFRES



<https://trends.google.fr/trends>

Stack Overflow Trends



Source : <https://insights.stackoverflow.com/trends>

Installation de l'environnement

- ✓ Bien vouloir consulter le support de cours sur l'installation de l'environnement de developpement de flutter sur Windows et Linux.
 - ✓ Faire reference au site de support officiel : <https://docs.flutter.dev/get-started/install>
-
- 1) Installation d'Android Studio avec la SDK android
 - 2) Telechargement et configuration de la SDK de flutter
 - 3) Flutter Doctor et configuration la la SDK d'android pour corriger les erreurs
 - 4) Installation de VS Code
 - 5) Installation du plugin flutter dans VS Code
 - 6) Creation d'un nouveau projet Flutter dans VS Code
 - 7) Creation d'un emulateur Android Studio (l'ADV d'android)
 - 8) Utilisation du telephone physique
 - 9) Structure d' un projet Flutter

STRUCTURE D' UN PROJET FLUTTER

The screenshot displays an IDE interface with the following components:

- EXPLORATEUR (Explorer):** Shows the project structure for 'PROJET01'. The 'lib' directory is expanded, showing 'main.dart' as the selected file. Other files include '.dart_tool', '.idea', 'android', 'ios', 'test', 'web', 'windows', '.gitignore', '.metadata', '.packages', 'analysis_options.yaml', 'projet01.iml', 'pubspec.lock', 'pubspec.yaml', and 'README.md'.
- main.dart:** The code editor shows the following Dart code:

```
lib > main.dart > MyApp > build
1  import 'package:flutter/material.dart';
2
   Run | Debug | Profile
3  void main() {
4      runApp(const MyApp());
5  }
6
7  class MyApp extends StatelessWidget {
8      const MyApp({Key? key}) : super(key: key);
9
10     // This widget is the root of your application.
11     @override
```
- PROBLÈMES (Problems):** The bottom panel shows the output of the 'flutter pub get' command:

```
flutter (projet01)
Creating project ....
Running "flutter pub get" in projet01..
.                                     4,7s
Wrote 97 files.

All done!
In order to run your application, type:

$ cd .
$ flutter run
```

Structure d'un projet Flutter

simple_application C:\Users\

> .dart_tool

> .idea

> android

> ios

✓ lib

main.dart

> test

.gitignore

.metadata

.packages

analysis_options.yaml

pubspec.lock

pubspec.yaml

README.md

simple_application.iml

> External Libraries

Scratches and Consoles

android: contient le code généré pour l'application android

ios: contient le code généré pour l'application ios

lib : contient le code Dart de l'application à développer

main.dart : point d'entrée de l'application à exécuter

test: contient les codes **Dart** pour tester l'application

pubspec.yaml: Un fichier utilisé pour déclarer les ressources et les dépendances relatives au projet comme images, polices, etc



Les bases de Dart

Conventions de nommage

- snake_case (ex : "home_screen.dart")
 - Noms de fichier
- UpperCamelCase (ex : "HomeScreen")
 - Classes
 - Constructeurs
- lowerCamelCase (ex : "toJson")
 - Méthodes
 - Variables
 - Constantes

Les types en Dart

- Tout ce qui est placé dans une variable **est un objet**, y compris les types "int", "double", "bool" et "null"

```
int a = 5;
```

- Si vous ne précisez pas un type à une variable, Dart va automatiquement le déduire

```
var a = 5;
```

- Le type "dynamic" équivaut à tout type

```
dynamic a = 5;  
a = "toto";
```

Les lists et les maps

- **List :**

Pas de type "Array" dans Dart

```
List<int> liste1 = List();  
List<int> liste2 = [];  
List<int> liste3 = [1, 2, 3];  
  
List<dynamic> liste4 = [1, 'Hello', 3.3];  
List liste5 = [4, "ma liste"];  
  
print(liste3[0]); // Affiche 1  
  
liste3.add(4);  
print(liste3.length); // Affiche 4
```

- **Map :**

```
Map<String, int> map1 = Map();  
Map<String, int> map2 = {'var1': 1, 'var2': 2};  
print(map2['var1']); // Affiche 1  
  
Map<String, dynamic> map3 = Map();  
map3['toto'] = 'titi';  
  
map3['tata'] = 3;  
  
Map map4 = Map(); // <=> Map<dynamic, dynamic>  
map4['toto'] = 'titi';  
map4[4] = 8;  
print(map4[4]); // Affiche 8
```

```
Map<String, int> map1 = Map();
Map<String, int> map2 = {'var1': 1, 'var2': 2};
print(map2['var1']); // Affiche 1

Map<String, dynamic> map3 = Map();
map3['toto'] = 'titi';
map3['tata'] = 3;

Map map4 = Map(); // <=> Map<dynamic, dynamic>
map4['toto'] = 'titi';
map4[4] = 8;
print(map4[4]); // Affiche 8
```

Les structures de contrôle

```
if (age < 14)
{
  print('Enfants');
} else if (age < 24)
{
  print('Adolescents');
} else {
  print('Adultes');
}
```

```
bool isPublic = true;
String visibility = isPublic ? 'public' : 'private';
```

Les boucles

```
List<String> students = ['s1', 's2', 's3'];  
for (var student in students) {  
    print(student);  
}  
  
for (var i = 0; i < students.length; i++) {  
    print(students[i]);  
}  
  
while (year < 2016) {  
    year += 1;  
}
```

```
// foreach((String) → void f) → void  
  
students.forEach((student) {  
    print(student);  
});  
  
students.asMap().forEach((index, student)  
{  
    print('$index: $student');  
});
```

Les fonctions

```
int somme(int a, int b) {  
  return a + b;  
}
```

```
int somme([int a=0, int b=0]) {  
  return a + b;  
}
```

```
int s= somme(1, 2);
```

```
int somme({required int a, required int b}) {  
  return a + b;  
}
```

```
int s= somme(a: 1, b: 2);
```

Les classes

```
class Person {  
    String name;  
    Person(this.name);  
}
```

```
Person(String name) {  
    this.name = name;  
}
```

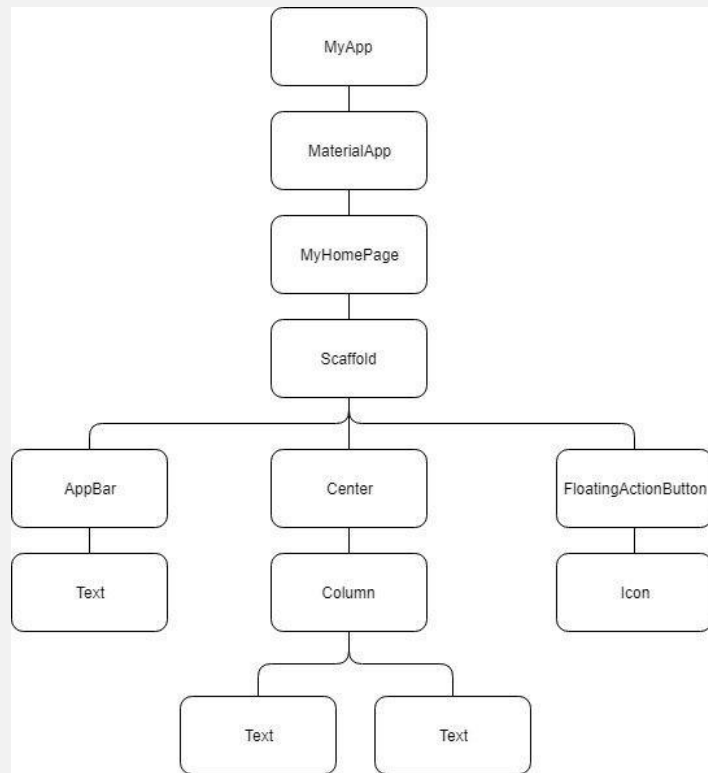
```
Person p = Person("toto");
```

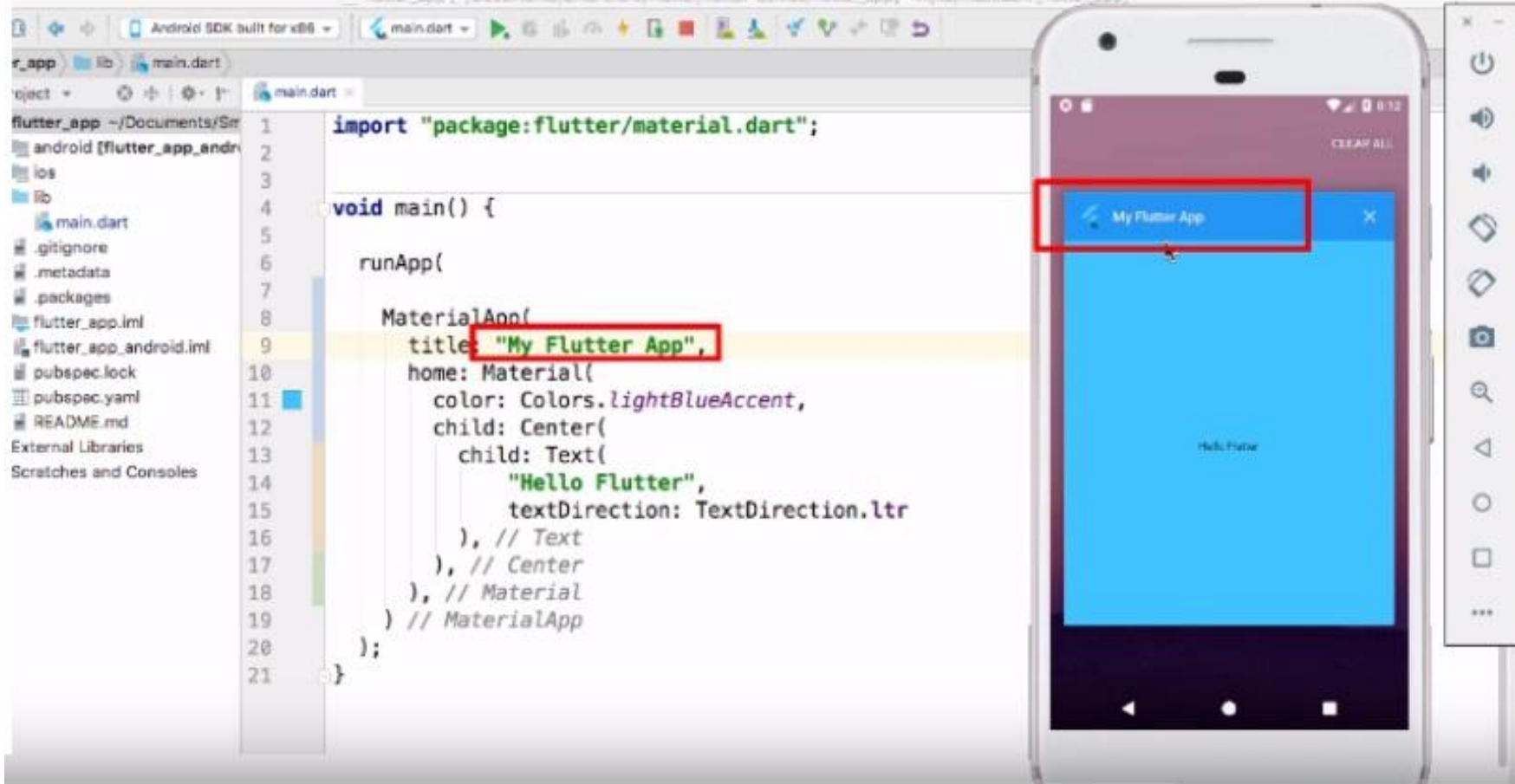

A decorative wavy line in dark blue and white, running vertically along the left edge of the slide.

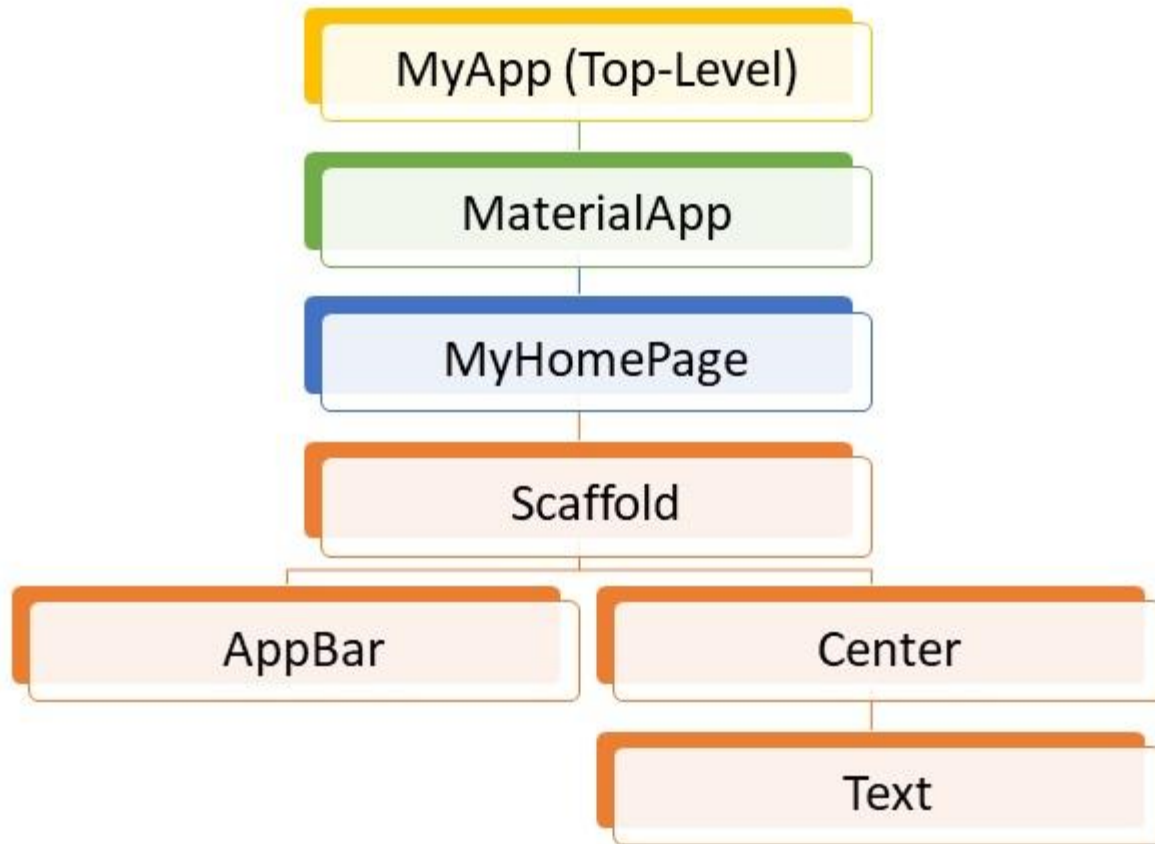
Introduction aux widgets

Qu'est-ce qu'un widget ?

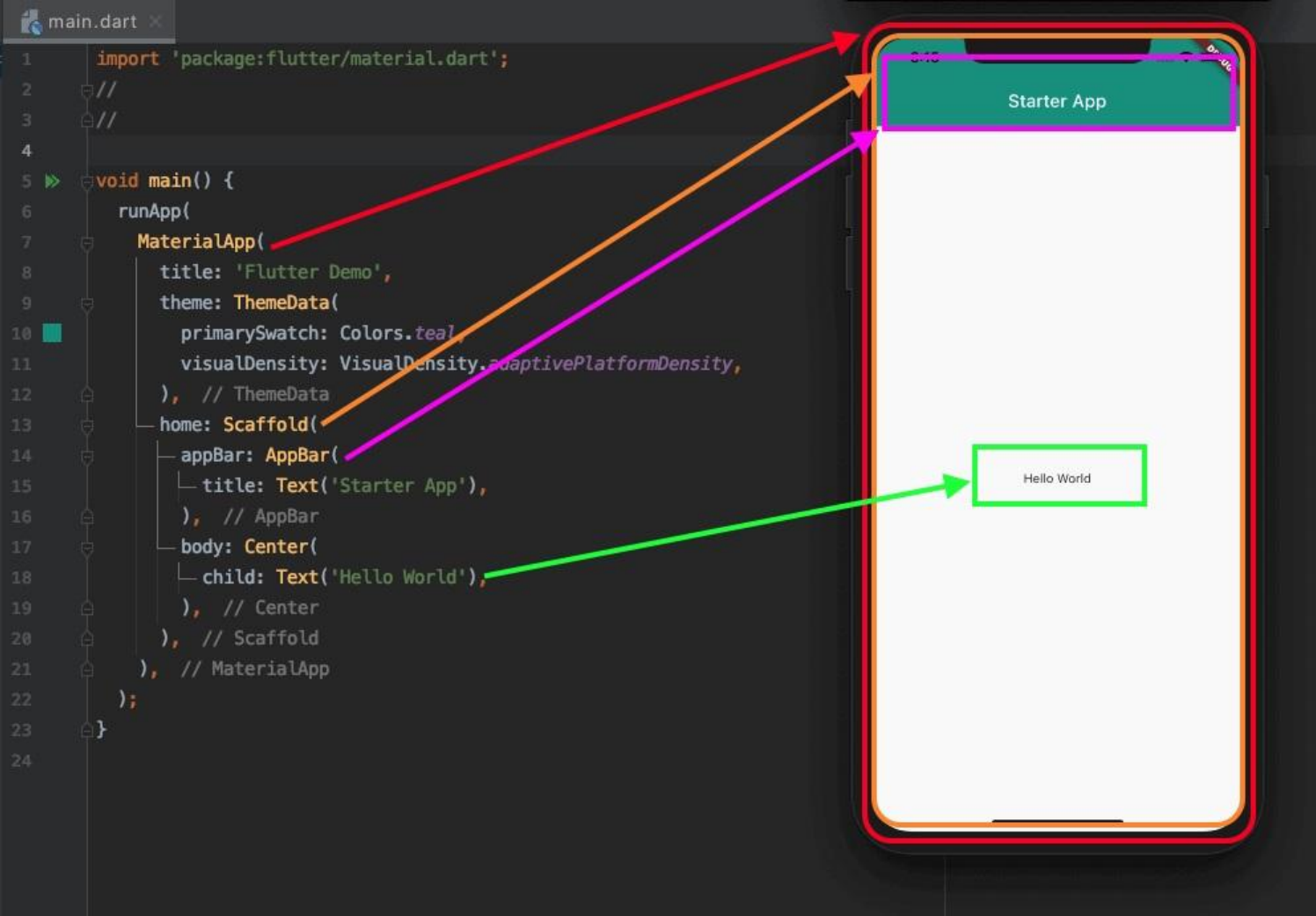
- **Dans Flutter, tout est widget !**
- Le rôle principal d'un widget est de faire un rendu via la méthode "**build()**"
- Les widgets sont organisés sous forme **d'arborescence**
- La méthode "runApp" accepte uniquement un type Widget en paramètre







WIDGETS DE BASES



Les widgets Stateless

- **Pas d'état** (pas de state)
- Comporte une seule méthode : "**Widget build(BuildContext context)**"
- **Ne peut pas être "re-builder" par lui-même**
- Cycle de vie :
 - Initialisation
 - Rendu (méthode build())

Les widgets Stateless

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return Container();  
  }  
}
```

Les widgets Stateful

- **Comporte un état**
- La méthode "build()" est déportée dans le state
- Une seule méthode : "**State<T> createState()**" où <T> correspond à la classe StatefulWidget
- Méthode "**build**" appelée à chaque **modification du state**
- State = **ensemble des attributs** de la classe State
- "**initState()**" : méthode pour initialisation le state

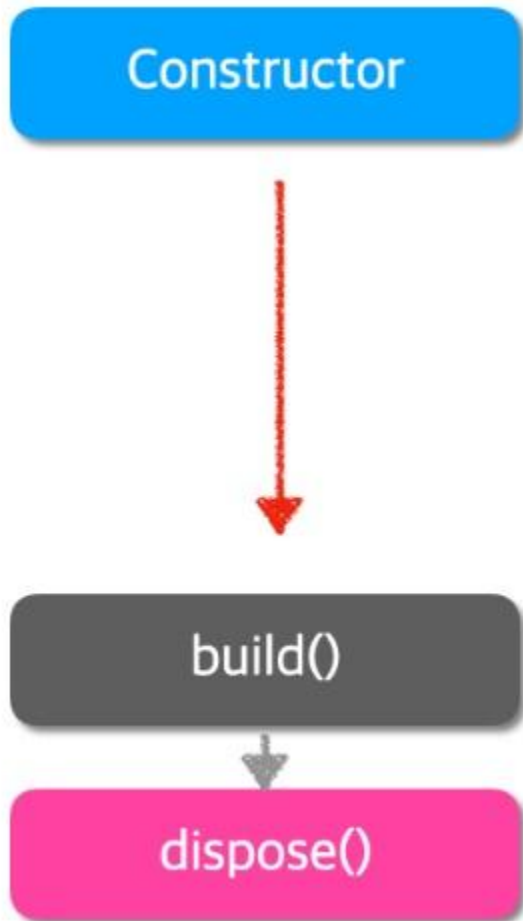
Les widgets Stateful

- **Comporte un état**
- La méthode "build()" est déportée dans le state
- Une seule méthode : "**State<T> createState()**" où <T> correspond à la classe StatefulWidget
- Méthode "**build**" appelée à chaque **modification du state**
- State = **ensemble des attributs** de la classe State
- "**initState()**" : méthode pour initialisation le state

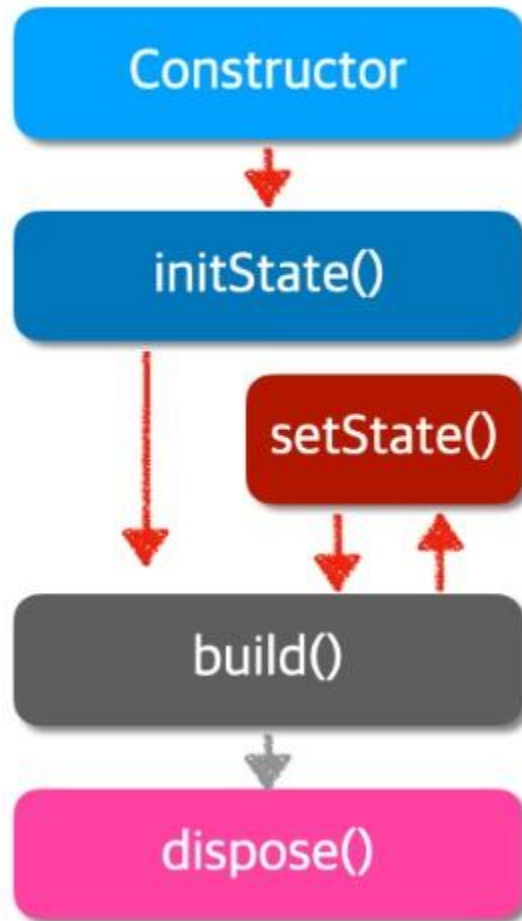
Les widgets Stateful

```
class MyApp extends StatefulWidget {  
  
  @override  
  _MyAppState createState() =>  
    _MyAppState();  
}  
  
class _MyAppState extends State<MyApp> {  
  late int _counter;  
  
  @override  
  void initState() {  
    _counter = 0;  
    super.initState();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return Text("Compteur $_counter");  
  }  
}
```

Stateless Widget



Stateful Widget



Le widget Text

```
Text(  
  "Compteur $_counter",  
  style: TextStyle(  
    fontWeight: FontWeight.bold,  
    fontSize: 16,  
    color: Colors.red,  
  ),  
);
```

A mobile application interface is shown on a smartphone screen. The screen is white with a black status bar at the top and a black home indicator bar at the bottom. In the center of the screen, the text "Compteur 0" is displayed in a bold, red font. A red "DEBUG" banner is visible in the top right corner of the screen.

Compteur 0

Le widget Image

pubspec.yaml

```
assets:  
  - assets/image/
```

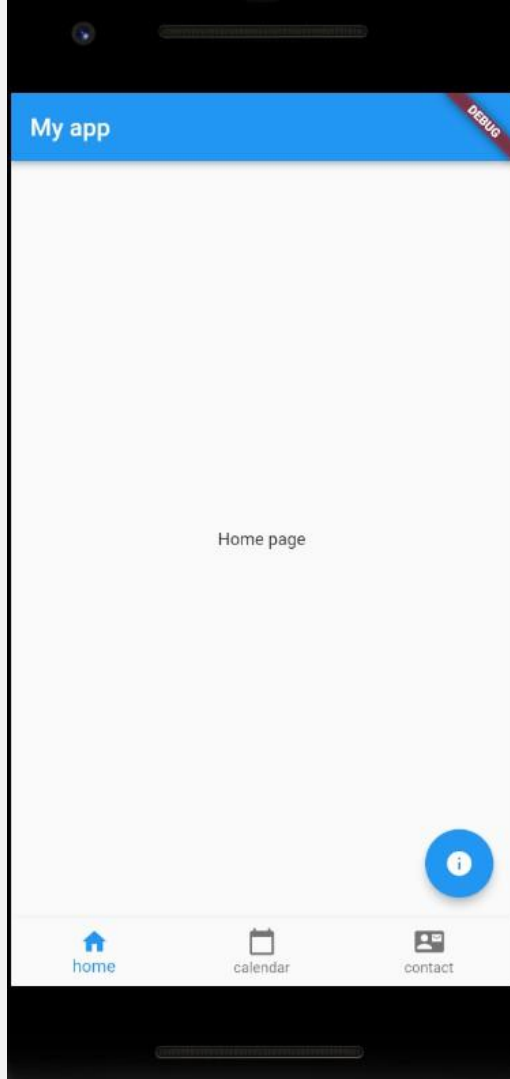
```
Center(  
  child: Image.asset("assets/image/  
my_image.jpg"),  
)
```

```
Image.network('https://picsum.photos/250?image=9')
```



Le widget Scaffold

```
Scaffold(  
  appBar: AppBar(  
    title: Text("My app"),  
  ),  
  body: Center(  
    child: Text("Home page"),  
  ),  
  floatingActionButton: FloatingActionButton(  
    onPressed: () {  
      print("clicked");  
    },  
    child: Icon(Icons.info),  
  ),  
  bottomNavigationBar: BottomNavigationBar(  
    items: [  
      BottomNavigationBarItem(  
        label: "home",  
        icon: Icon(Icons.home)  
      ),  
      BottomNavigationBarItem(  
        label: "calendar",  
        icon: Icon(Icons.calendar_today)  
      ),  
      BottomNavigationBarItem(  
        label: "contact",  
        icon: Icon(Icons.contact_mail)  
      ),  
    ],  
  ),  
)
```

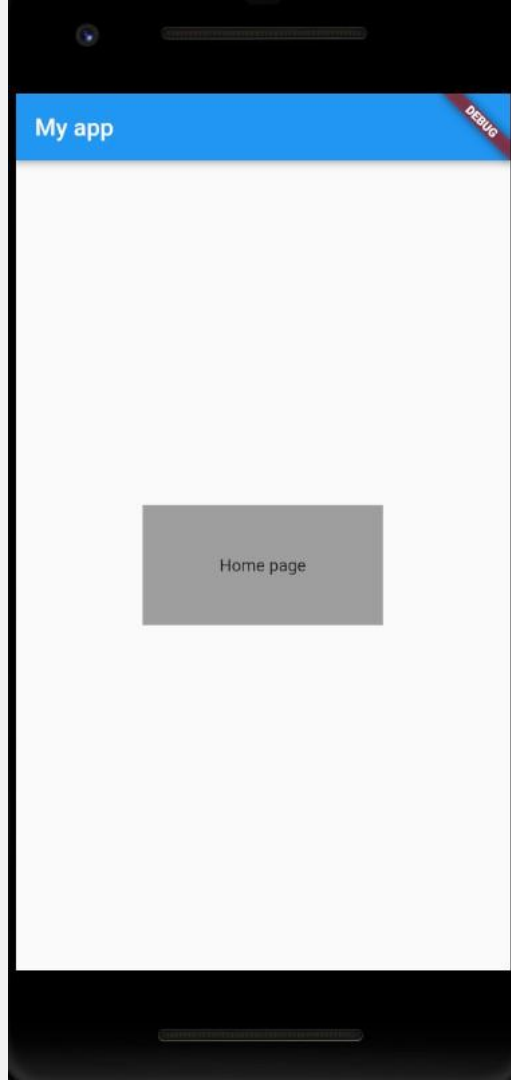




Les widgets de layout

Le widget container

```
Center(  
  child: Container(  
    height: 100,  
    width: 200,  
    color: Colors.grey,  
    margin: EdgeInsets.all(20),  
    child: Center(  
      child: Text("Home page")  
    ),  
  ),  
)
```



Le widget padding

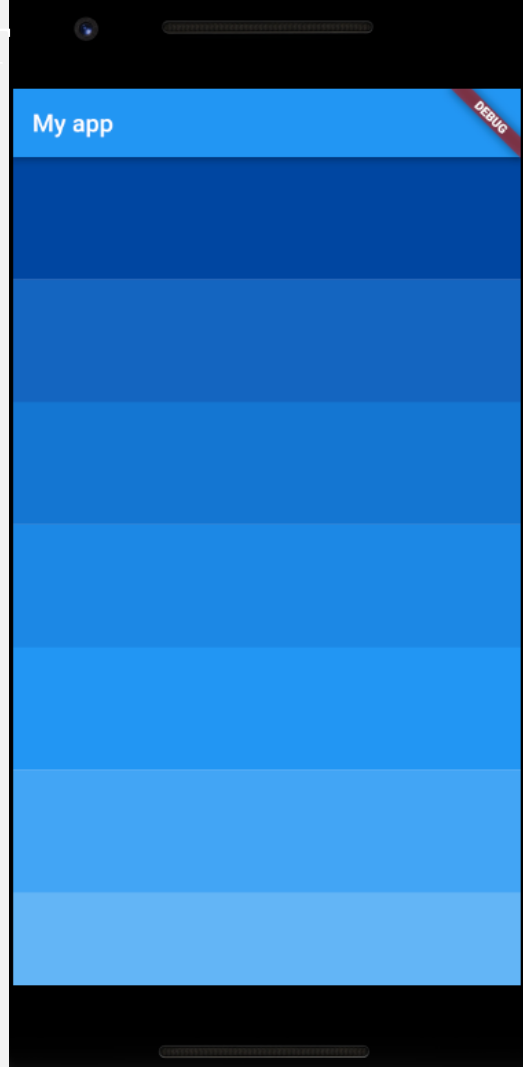
```
Center(  
  child: Container(  
    height: 100,  
    width: 200,  
    color: Colors.grey,  
    margin: EdgeInsets.all(20),  
    child: Padding(  
      padding: const EdgeInsets.all(8.0),  
      child: Container(  
        color: Colors.yellow,  
        child: Center(  
          child: Text("Home page")  
        ),  
      ),  
    ),  
  ),  
)
```



Le widget column

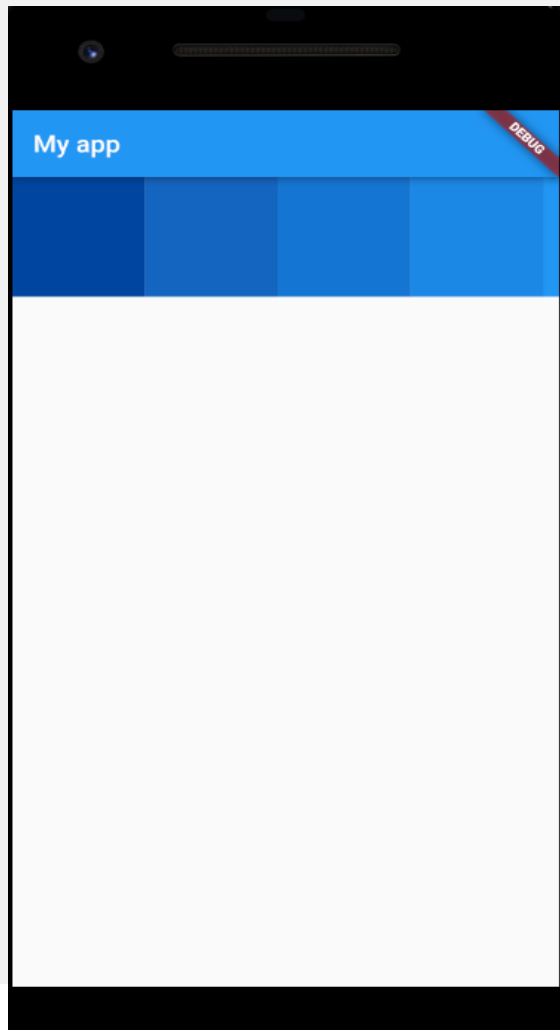
```
SingleChildScrollView(  
  child: Column(  
    children: [  
      Container(  
        height: 100,  
        color: Colors.blue[900],  
      ),  
      Container(  
        height: 100,  
        color: Colors.blue[800],  
      ),  
      Container(  
        height: 100,  
        color: Colors.blue[700],  
      ),  
      Container(  
        height: 100,  
        color: Colors.blue[600],  
      ),  
      Container(  
        height: 100,  
        color: Colors.blue[500],  
      ),  
    ],  
  ),  
)
```

Le widget column



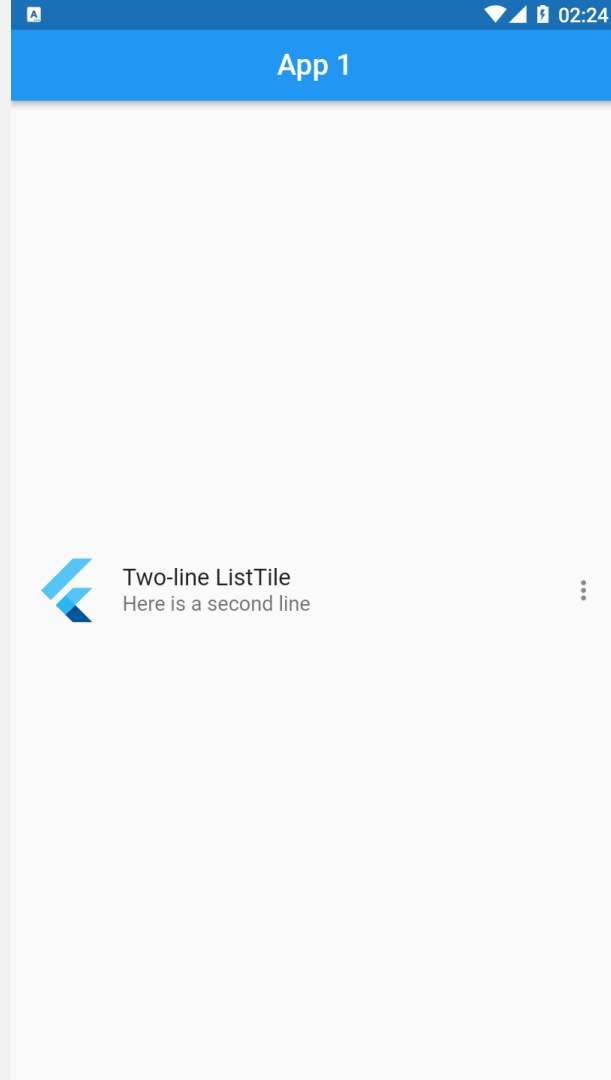
Le widget row

```
SingleChildScrollView(  
  scrollDirection: Axis.horizontal,  
  child: Row(  
    children: [  
      Container(  
        width: 100,  
        height: 100,  
        color: Colors.blue[900],  
      ),  
      Container(  
        width: 100,  
        height: 100,  
        color: Colors.blue[800],  
      ),  
      Container(  
        width: 100,  
        height: 100,  
        color: Colors.blue[700],  
      ),  
      Container(  
        width: 100,  
        height: 100,  
        color: Colors.blue[600],  
      ),  
    ],  
  ),  
)
```



Le widget ListTile

```
ListTile(  
  leading: FlutterLogo(size: 56.0),  
  title: Text('Two-line ListTile'),  
  subtitle: Text('Here is a second  
line'),  
  trailing: Icon(Icons.more_vert),  
)
```





Les widgets relatifs aux listes

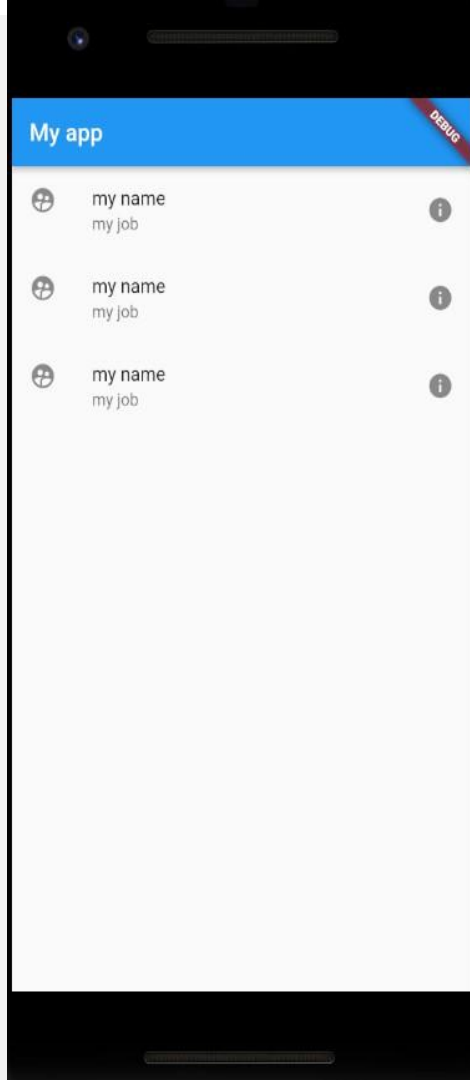
Le widget ListView

```
ListView(  
  children: [  
    Container(  
      width: 100,  
      height: 100,  
      color: Colors.blue[900],  
    ),  
    Container(  
      width: 100,  
      height: 100,  
      color: Colors.blue[800],  
    ),  
    Container(  
      width: 100,  
      height: 100,  
      color: Colors.blue[700],  
    ),  
    Container(  
      width: 100,  
      height: 100,  
      color: Colors.blue[600],  
    ),  
  ],  
)
```



Le widget ListTile

```
ListView(  
  children: [  
    ListTile(  
      leading: Icon(Icons.supervised_user_circle),  
      title: Text("my name"),  
      subtitle: Text("my job"),  
      trailing: Icon(Icons.info),  
    ),  
    ListTile(  
      leading: Icon(Icons.supervised_user_circle),  
      title: Text("my name"),  
      subtitle: Text("my job"),  
      trailing: Icon(Icons.info),  
    ),  
    ListTile(  
      leading: Icon(Icons.supervised_user_circle),  
      title: Text("my name"),  
      subtitle: Text("my job"),  
      trailing: Icon(Icons.info),  
    ),  
  ],  
)
```

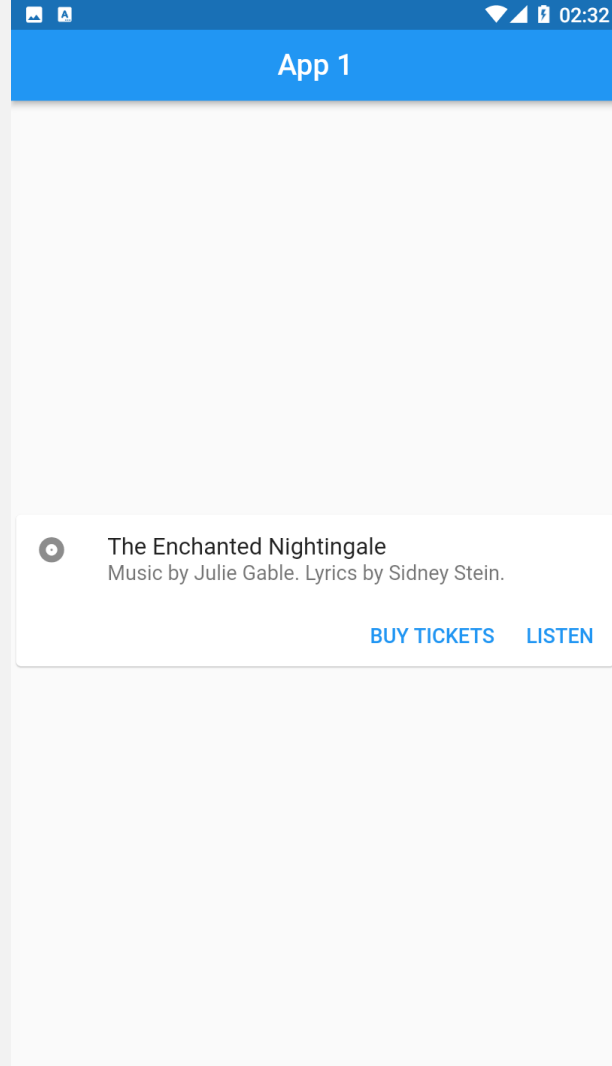


A decorative wavy line in white and dark blue runs vertically along the left side of the image.

Les widgets d'Informations

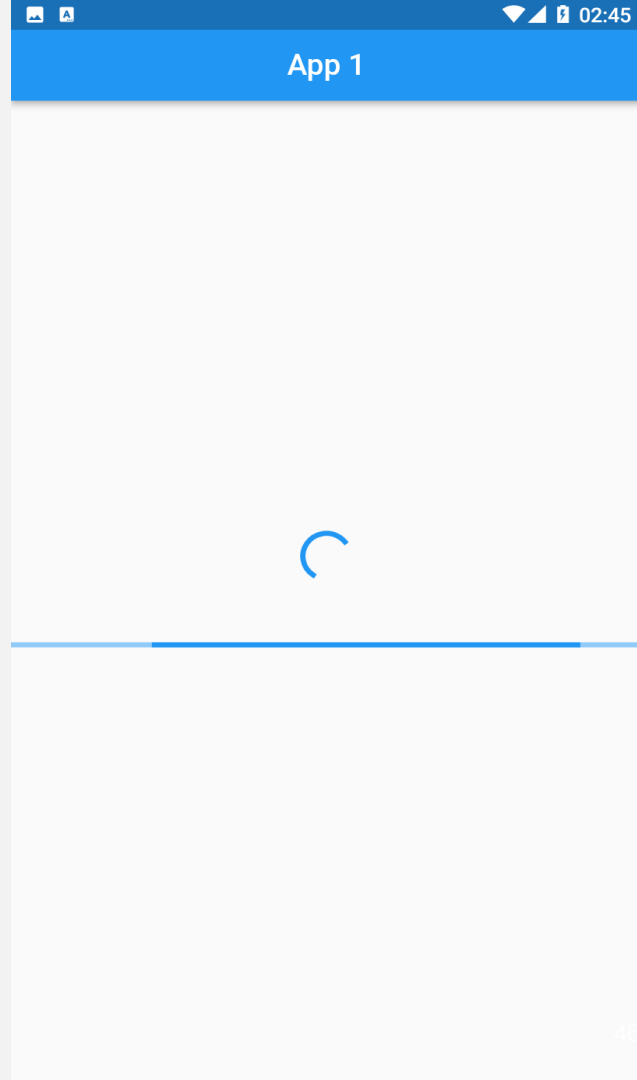
Card

```
Card(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: <Widget>[
      const ListTile(
        leading: Icon(Icons.album),
        title: Text('The Enchanted Nightingale'),
        subtitle: Text('Music by Julie Gable. Lyrics by
                      Sidney Stein.'),
      ),
      Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: <Widget>[
          TextButton(
            child: const Text('BUY TICKETS'),
            onPressed: () { /* ... */ },
          ),
          const SizedBox(width: 8),
          TextButton(
            child: const Text('LISTEN'),
            onPressed: () { /* ... */ },
          ),
          const SizedBox(width: 8),
        ],
      ),
    ],
  ),
)
```



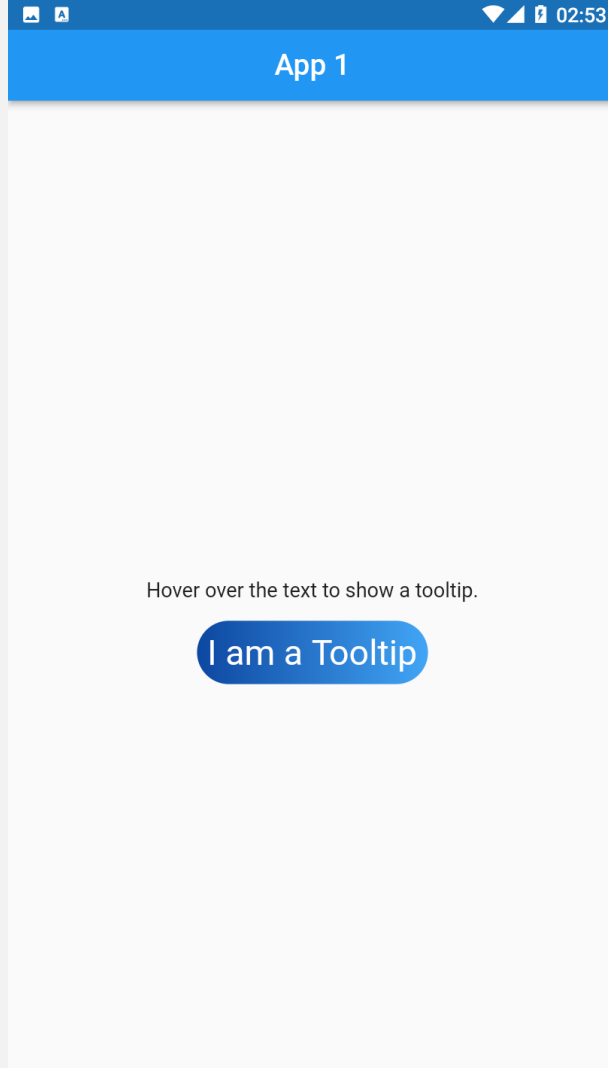
ProgressIndicator

```
Column(  
  mainAxisAlignment:  
    MainAxisAlignment.center,  
  children: [  
    CircularProgressIndicator(),  
    SizedBox(height: 50,),  
    LinearProgressIndicator()  
  ],  
)
```



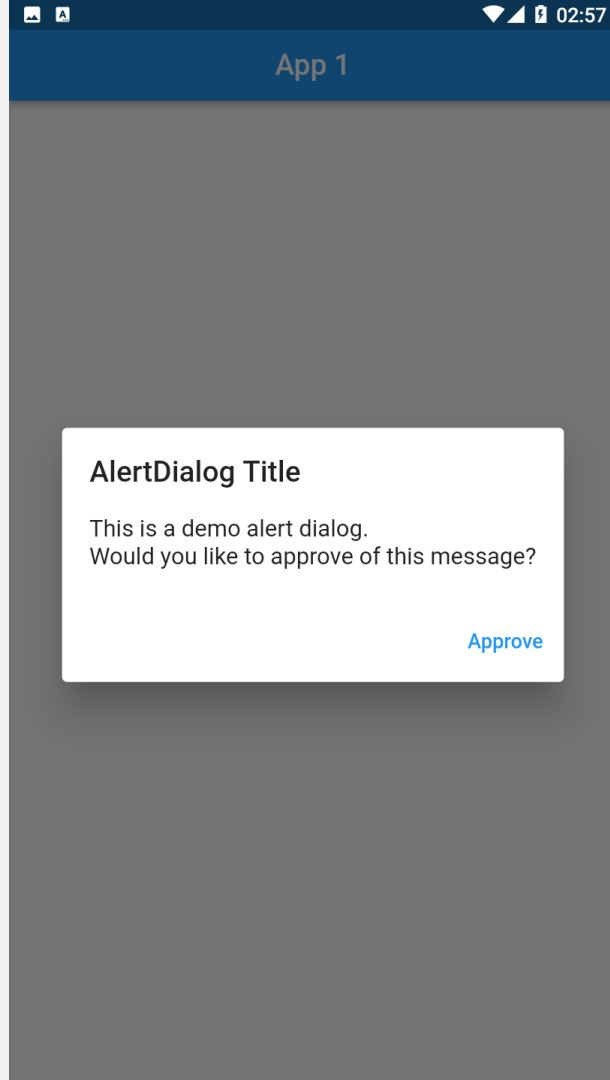
Tooltip

```
Tooltip(  
  message: 'I am a Tooltip',  
  decoration: BoxDecoration(  
    borderRadius: BorderRadius.circular(25),  
    gradient:  
      LinearGradient(  
        colors: <Color>[  
          Colors.blue.shade900,  
          Colors.blue[400]!  
        ],  
      ),  
  ),  
  height: 50,  
  padding: const EdgeInsets.all(8.0),  
  preferBelow: true,  
  textStyle: const TextStyle(  
    fontSize: 24,  
    color: Colors.white  
  ),  
  showDuration: Duration(seconds: 3),  
  child: Text('Hover over the text to show a  
tooltip.'),  
)
```



AlertDialog

```
Future<void> _showMyDialog() async {  
  return showDialog<void>(  
    context: context,  
    barrierDismissible: false, // user must tap button!  
    builder: (BuildContext context) {  
      return AlertDialog(  
        title: const Text('AlertDialog Title'),  
        content: SingleChildScrollView(  
          child: ListBody(  
            children: const <Widget>[  
              Text('This is a demo alert dialog.'),  
              Text('Would you like to approve of this  
                message?'),  
            ],  
          ),  
        ),  
        actions: <Widget>[  
          TextButton(  
            child: const Text('Approve'),  
            onPressed: () {  
              Navigator.of(context).pop();  
            },  
          ),  
        ],  
      );  
    },  
  );  
}
```

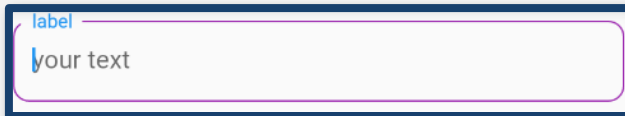




Les widgets d'Input et de sélection

TextField

```
TextField(  
  decoration: InputDecoration(  
    border: OutlineInputBorder(  
      borderRadius:  
        BorderRadius.all(Radius.circular(10.0)),  
    ),  
    focusedBorder: OutlineInputBorder(  
      borderSide: BorderSide(color: Colors.purple),  
      borderRadius:  
        BorderRadius.all(Radius.circular(10.0)),  
    ),  
    hintText: "your text",  
    label: Text("label"),  
    fillColor: Colors.white70  
  ),  
)
```



```
border:  
  UnderlineInputBorder(),
```



TextField

```
TextField(  
  obscureText: true,  
  decoration: InputDecoration(  
    border: OutlineInputBorder(),  
    labelText: 'Password',  
  ),  
),
```



```
TextEditingController controller = TextEditingController();
```

```
TextField(  
  controller: controller,  
  obscureText: true,  
  decoration: InputDecoration(  
    border: OutlineInputBorder(),  
    labelText: 'Password',  
  ),  
),
```



Form

```
final _formKey =
GlobalKey<FormState>();
```

```
Form(
  key: _formKey,
  child: Column(
    children: [],
  )
)
```

```
TextFormField(
  decoration: const InputDecoration(
    border: OutlineInputBorder(),
    labelText: 'Email',
    hintText: 'Enter Email'
  ),
  autovalidateMode:
AutovalidateMode.onUserInteraction,
  validator: (value) {
    if (value!.isEmpty) {
      return "* Required";
    } else if (!value.isValidEmail())
    {
      return "Check your email";
    } else
      return null;
  },
)
```

```
ElevatedButton(
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Processing
Data')),
      );
    }
  },
  child: const Text('Submit'),
)
```



Form

```

Form(
  key: _formKey,
  child: Column(
    children: <Widget>[
      TextFormField(
        decoration: const InputDecoration(
          border: OutlineInputBorder(),
          labelText: 'Email',
          hintText: 'Enter Email'
        ),
        validator: (value) {
          if (value!.isEmpty) {
            return "* Required";
          } else if (!value.isValidEmail()) {
            return "Check your email";
          } else
            return null;
        },
      ),
      const  SizedBox(height: 16,),
      TextFormField(
        decoration: const InputDecoration(
          border: OutlineInputBorder(),
          labelText: 'Password',
          hintText: 'Enter secure password'
        ),
        obscureText: true,
        validator: (value) {
          if (value!.isEmpty) {
            return "* Required";
          } else if (value.length < 6) {
            return "Password should be atleast 6 characters";
          } else if (value.length > 15) {
            return "Password should not be greater than 15 characters";
          } else
            return null;
        },
      ),
      const  SizedBox(height: 16,),
      ElevatedButton(
        onPressed: () {
          if (_formKey.currentState!.validate())
          {
            ScaffoldMessenger.of(context).showSnackBar(
              const  SnackBar(content:
                Text('Processing Data')),
            );
          }
        },
        child: const Text('Submit'),
      ),
    ],
  ),
)

```

Form

App 1

Email

Password

Submit

App 1

Email

Check your email

Password

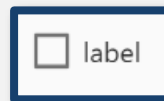
Password should be atleast 6 characters

Submit

Processing Data

Checkbox

```
Row(  
  children: [  
    Checkbox(  
      checkColor: Colors.white,  
      value: isChecked,  
      onChanged: (bool? value) {  
        setState(() {  
          isChecked = value!;  
        });  
      },  
    ),  
    const Text("label")  
  ],  
)
```



Radio

```
Text("groupe 1"),
ListTile(
  title: const Text('A'),
  leading: Radio<String>(
    value: "a",
    groupValue: __character,
    onChanged: (String? value) {
      setState(() {
        __character = value!;
      });
    },
  ),
),
ListTile(
  title: const Text('B'),
  leading: Radio<String>(
    value: "b",
    groupValue: __character,
    onChanged: (String? value) {
      setState(() {
        __character = value!;
      });
    },
  ),
),
```

```
Text("groupe 2"),
ListTile(
  title: const Text('C'),
  leading: Radio<String>(
    value: "c",
    groupValue: __character2,
    onChanged: (String? value) {
      setState(() {
        __character2 = value!;
      });
    },
  ),
),
ListTile(
  title: const Text('D'),
  leading: Radio<String>(
    value: "d",
    groupValue: __character2,
    onChanged: (String? value) {
      setState(() {
        __character2 = value!;
      });
    },
  ),
),
```

```
ListTile(
  title: const Text('E'),
  leading: Radio<String>(
    value: "e",
    groupValue: __character2,
    onChanged: (String? value) {
      setState(() {
        __character2 = value!;
      });
    },
  ),
),
```





Les widgets de Bouton

ElevatedButton et OutlinedButton

```
ElevatedButton(  
  onPressed: () {},  
  style: ElevatedButton.styleFrom(  
    shape: RoundedRectangleBorder(  
      borderRadius: BorderRadius.circular(30.0),  
    ),  
  ),  
  child: const Text(' Elevated Button', style: TextStyle(fontSize:  
18),),  
,  
const SizedBox(height: 12,),  
ElevatedButton(  
  onPressed: () {},  
  child: const Text(' Elevated Button', style: TextStyle(fontSize:  
18),),  
,
```

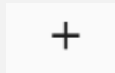
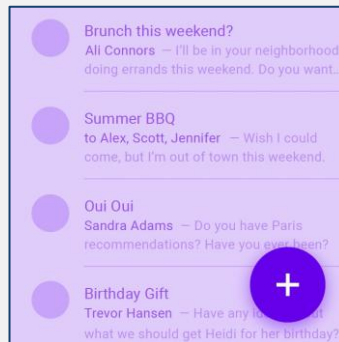
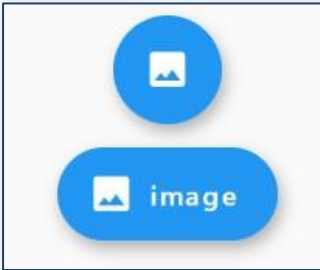
```
OutlinedButton(  
  onPressed: () {  
    debugPrint('Received click');  
  },  
  child: const Text('Click Me'),  
)
```

A blue rounded rectangular button with a subtle drop shadow, containing the text "Elevated Button" in white.A blue rounded rectangular button with a subtle drop shadow, containing the text "Elevated Button" in white.A white rectangular button with a thin blue border and a subtle drop shadow, containing the text "Click Me" in blue.

FloatingActionButton et IconButton

```
const SizedBox(height: 12,),  
FloatingActionButton(  
  onPressed: () {},  
  child: const  
    Icon(Icons.image),  
),  
const SizedBox(height: 12,),  
FloatingActionButton.extended(  
  onPressed: () {},  
  label: const Text("image"),  
  icon: const Icon(Icons.image),  
),
```

```
IconButton(  
  onPressed: () {},  
  icon: const Icon(Icons.add)  
),
```

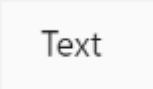
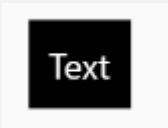


TextButton et InkWell

```
TextButton(  
  style: TextButton.styleFrom(  
    textStyle: const TextStyle(fontSize:  
20),  
  ),  
  onPressed: () {},  
  child: const Text('Text'),  
)
```

```
InkWell(  
  onTap: () {},  
  child: const  
Text('Text')  
)
```

```
InkWell(  
  onTap: () {},  
  child: Container(  
    color: Colors.black,  
    child: const Text(  
      'Text',  
      style: TextStyle(color: Colors.white),  
    )  
  )  
)
```

A white rectangular button with rounded corners and a blue border. The word "Text" is centered in a blue font.A white rectangular button with rounded corners and a light gray border. The word "Text" is centered in a light gray font.A white rectangular button with rounded corners and a black background. The word "Text" is centered in a white font.



Les routes

Navigation vers une nouvelle page

```
ElevatedButton(  
  onPressed: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) => SecondRoute()),  
      );  
  },  
  child: const Text("click")  
)
```



Navigation avec des routes nommées

```
MaterialApp(  
  title: 'Named Routes Demo',  
  initialRoute: '/',  
  routes: {  
    '/': (context) => const FirstScreen(), widget.  
    '/second': (context) => const SecondScreen(),  
  },  
)  
)
```

```
ElevatedButton(  
  onPressed: () {  
    Navigator.pushNamed(context,  
      '/second');  
  },  
  child: const Text("click")  
)
```



Revenir à la page précédente

```
ElevatedButton(  
  onPressed: () {  
    Navigator.pop(context);  
  },  
  child: const Text("Back")  
)
```



Passage de données entre les Widgets

Routes nommées

Envoyer

```
Navigator.of(context).pushNamed("/second", arguments: "argument 1");
```

Récupérer

```
final args =  
ModalRoute.of(context)!.settings.arguments;
```

Routes non nommées

Envoyer

```
MyWidget(  
  param1: "text 1",  
  param2: "Text 2",  
)
```

Récupérer

```
class MyWidget extends StatefulWidget {  
  String param1;  
  String param2;  
  MyWidget({  
    required this.param1,  
    required this.param2  
  });  
  @override  
  _MyWidgetState createState() =>  
  _MyWidgetState();  
}
```

Accéder StatefulWidget

```
Text(  
  widget.param1  
)
```

Accéder StatelessWidget

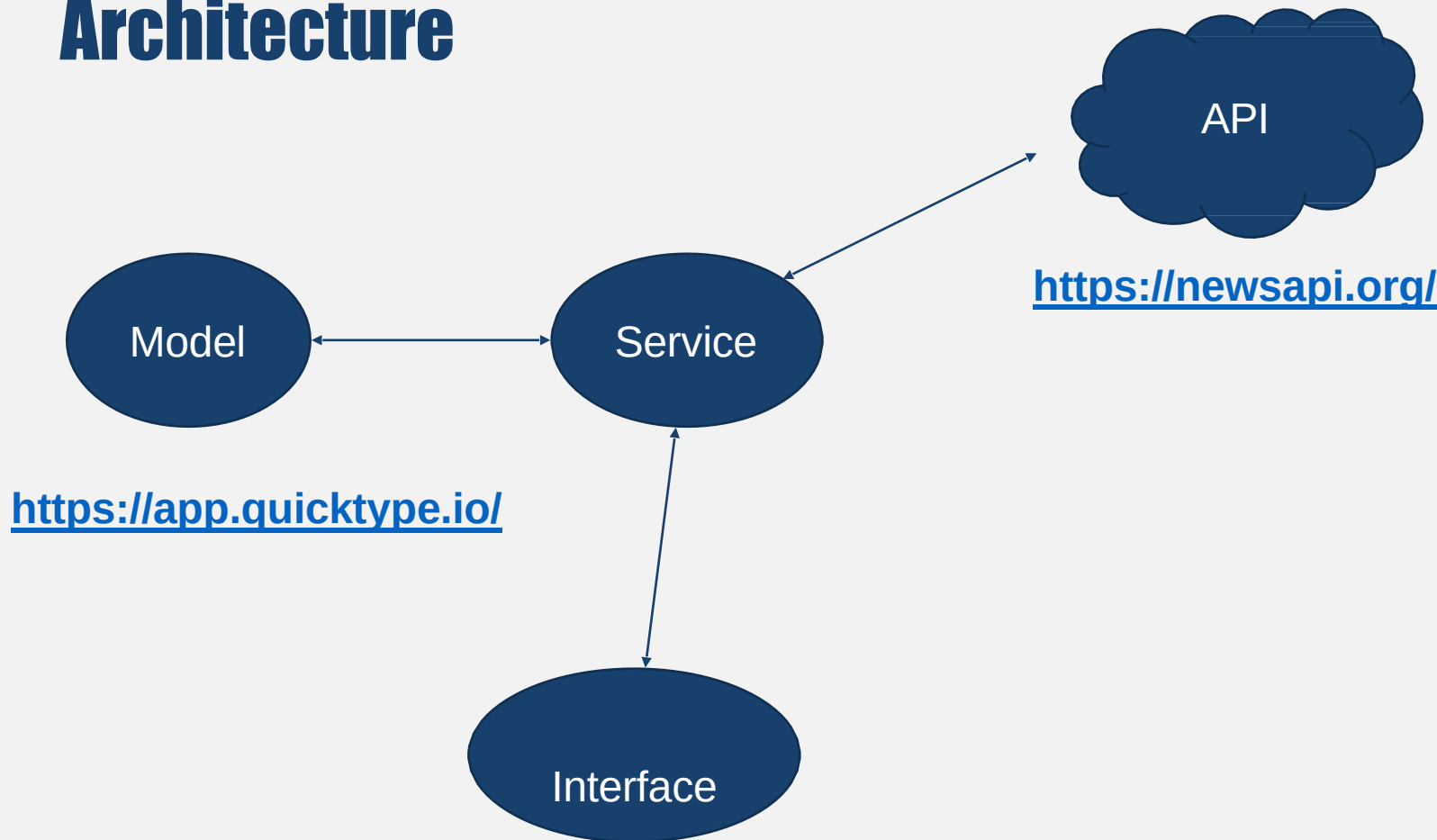
```
Text(  
  param1  
)
```



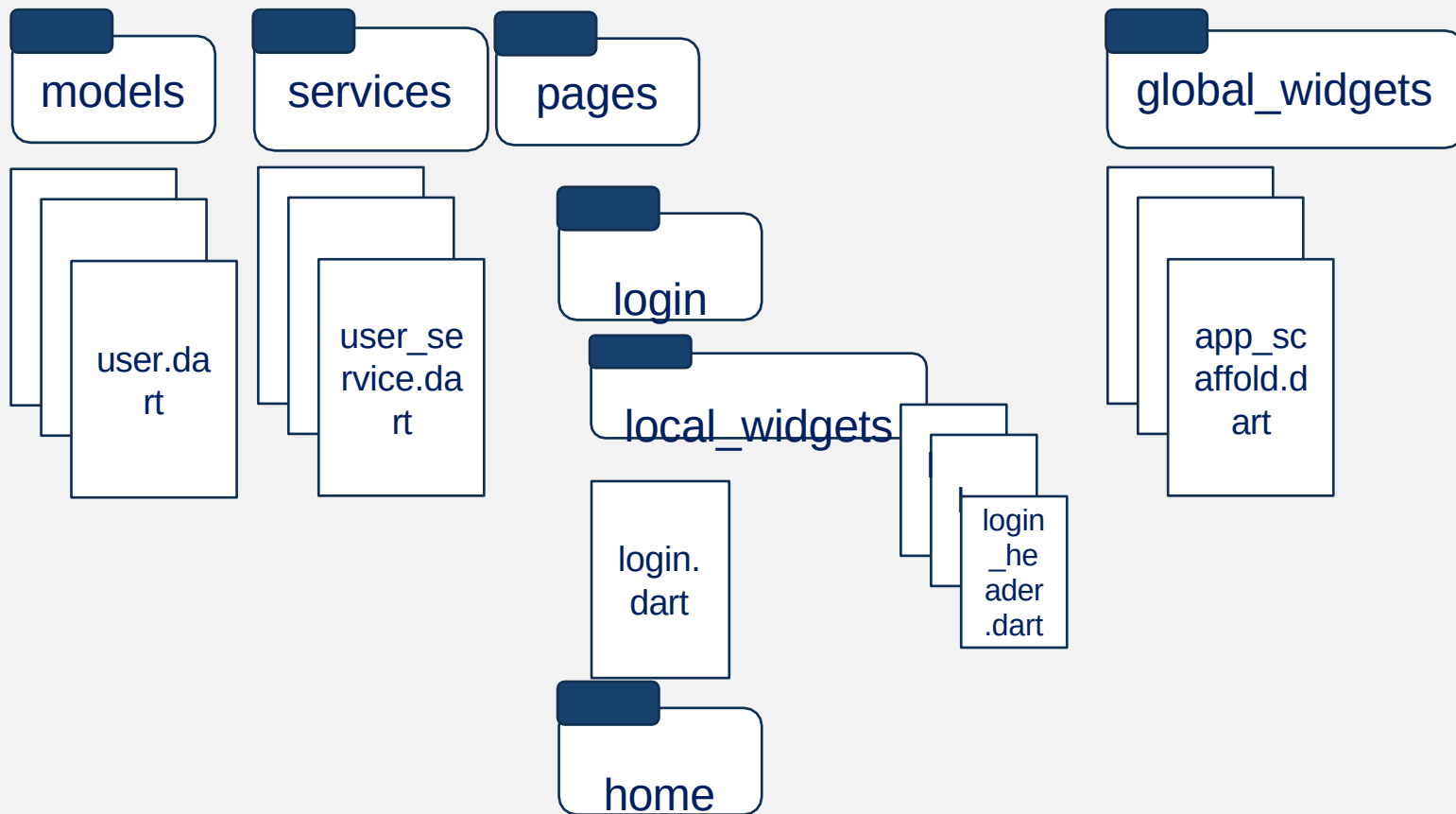


Consommation des web services

Architecture



Architecture



Model

```
class Article {  
    Article({  
        this.source,  
        this.author,  
        this.title,  
        this.description,  
        this.url,  
        this.urlToImage,  
        this.publishedAt,  
        this.content,  
    });  
  
    final Source? source;  
    final String? author;  
    final String? title;  
    final String? description;  
    final String? url;  
    final String? urlToImage;  
    final DateTime? publishedAt;  
    final String? content;  
}
```



Model

```
factory Article.fromJson(Map<String, dynamic> json) => Article(  
  source: json["source"] == null ?  
    null :  
    Source.fromJson(json["source"]),  
  author: json["author"] == null ? null : json["author"],  
  title: json["title"] == null ? null : json["title"],  
  description: json["description"] == null ?  
    null :  
    json["description"],  
  url: json["url"] == null ? null : json["url"],  
  urlToImage: json["urlToImage"] == null ?  
    null :  
    json["urlToImage"],  
  publishedAt: json["publishedAt"] == null ?  
    null :  
    DateTime.parse(json["publishedAt"]),  
  content: json["content"] == null ? null : json["content"],  
);
```

Model

```
Map<String, dynamic> toJson() => {  
  "source": source == null ? null : source!.toJson(),  
  "author": author == null ? null : author,  
  "title": title == null ? null : title,  
  "description": description == null ? null : description,  
  "url": url == null ? null : url,  
  "urlToImage": urlToImage == null ? null : urlToImage,  
  "publishedAt": publishedAt == null ?  
    null :  
    publishedAt!.toIso8601String(),  
  "content": content == null ? null : content,  
};
```



Model

```
Data dataFromJson(String str) => Data.fromJson(json.decode(str));  
String dataToJson(Data data) => json.encode(data.toJson());
```



service

installer la dépendance

[https://pub.dev/packages/
http](https://pub.dev/packages/http)

pubspec.yaml

```
dependencies:  
  http: ^0.13.4
```

run



flutter pub get

```
import 'package:http/http.dart' as  
http;
```



service

```
class NewsService{
    Future<Data> getAll() async {
        var url = Uri.parse('https://newsapi.org/v2/top-headlines?
country=us&category=business&apiKey=e0ac43cc665a48aeb762ed7dae8139e9'
);
        var response = await http.get(url,);
        print('Response status: ${response.statusCode}');
        print('Response body: ${response.body}');
        return dataFromJson(response.body);
    }
}
```

```
Map<String, String> queryParameters = {
    "country": "us",
    "category": "business",
    "apiKey":
    "e0ac43cc665a48aeb762ed7dae8139e9",
};
var url = Uri(
    scheme: 'https',
    host: 'newsapi.org',
    path: '/v2/top-headlines',
    queryParameters: queryParameters,
);
```



Interface

```
NewsService newsService = NewsService();
```

```
newsService.getAll().then((value) {  
    print(value.articles!.first.title);  
});
```

OU

```
Data data = await newsService.getAll();
```



Interface

```
NewsService newsService= NewsService();
```

```
FutureBuilder(  
  future: newsService.getAll(),  
  builder: (BuildContext context, AsyncSnapshot snapshot) {  
    if (snapshot.hasData) {  
      return OnSucces(  
        data: snapshot.data,  
      );  
    } else if (snapshot.hasError) {  
      return OnError(  
        error: snapshot.error.toString(),  
      );  
    } else {  
      return const Waiting();  
    }  
  },  
)
```



Interface

```
class OnError extends StatelessWidget {  
  final String error;  
  OnError({  
    Key? key,  
    required this.error  
  }) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children:<Widget>[  
          const Icon(  
            Icons.error_outline,  
            color: Colors.red,  
            size: 60,  
          ),  
          Padding(  
            padding: const EdgeInsets.only(top: 16),  
            child: Text('Error: ${error}'),  
          )  
        ],  
      ),  
    );  
  }  
}
```



Interface

```
class Waiting extends StatelessWidget {  
  const Waiting({  
    Key? key,  
  }) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: const <Widget>[  
          SizedBox(  
            width: 60,  
            height: 60,  
            child: CircularProgressIndicator(),  
          ),  
          Padding(  
            padding: EdgeInsets.only(top: 16),  
            child: Text('Awaiting result...'),  
          )  
        ],  
      ),  
    );  
  }  
}
```

Interface

```
class OnSucces extends StatelessWidget {  
  final Data data;  
  OnSucces({  
    Key? key,  
    required this.data  
  }) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: <Widget>[  
          const Icon(  
            Icons.check_circle_outline,  
            color: Colors.green,  
            size: 60,  
          ),  
          Padding(  
            padding: const EdgeInsets.only(top: 16),  
            child: Text('Result: ${  
              {data.articles!.first.title}',  
            )  
          ),  
        ],  
      ),  
    );  
  }  
}
```

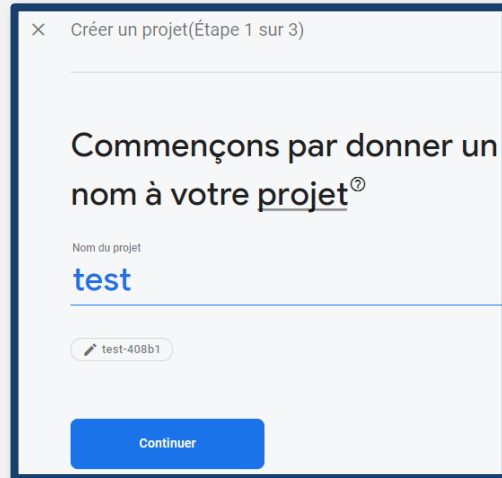




Firebase Cloud Firestore

Créer un projet Firebase

- Accédez à la [console Firebase](#) et cliquez sur Ajouter un projet.
- Saisissez un nom de projet et cliquez sur Continuer .



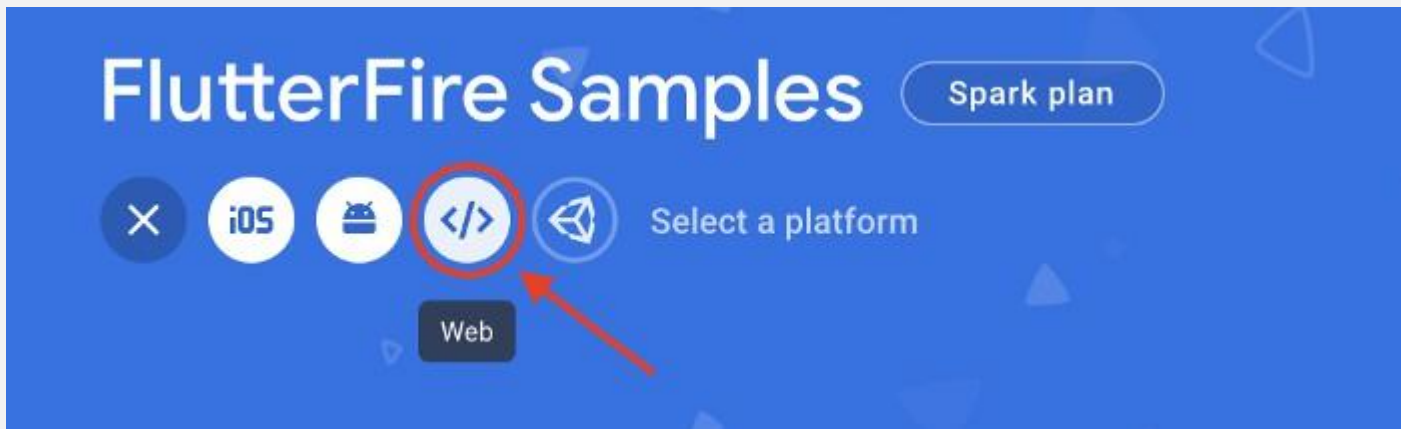
Intégration de Firebase avec Android, iOS et Web

Sélectionnez l'icône de la plate-forme souhaitée dans le tableau de bord Firebase.



onfiguration de la plate-forme Web

- Sélectionnez l'icône Web sur le tableau de bord.



Configuration de la plate-forme Web

- Entrez le pseudo de l'application et cliquez sur Enregistrer l'application

× Ajouter Firebase à votre application Web

1

 Enregistrer l'application

Pseudo de l'application ?

my blog

☐ Configurez également **Firebase Hosting** pour cette application.
[En savoir plus](#) 

Hosting peut être configuré plus tard. Vous pouvez commencer à l'utiliser gratuitement à tout moment.

Enregistrer l'application

Configuration de la plate-forme Web

- Ajouter le SDK Firebase

2 Ajouter le SDK Firebase

☐ Utiliser npm ⓘ ☒ Utiliser une balise <script> ⓘ

Copiez et collez ces scripts en bas de votre balise <body> avant d'utiliser les services Firebase :

```
<script type="module">
  // Import the functions you need from the SDKs you need
  import { initializeApp } from "https://www.gstatic.com/firebasejs/9.6.2/firebase-app.js";
  import { getAnalytics } from "https://www.gstatic.com/firebasejs/9.6.2/firebase-analytics.js";
  // TODO: Add SDKs for Firebase products that you want to use
  // https://firebase.google.com/docs/web/setup#available-libraries

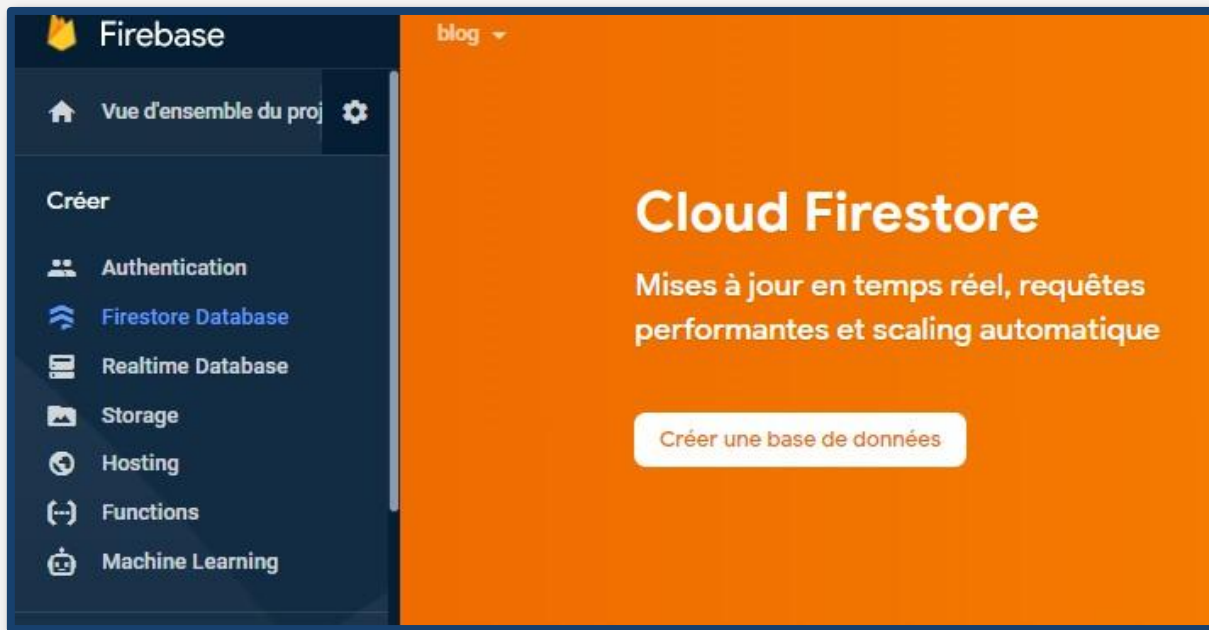
  // Your web app's Firebase configuration
  // For Firebase JS SDK v7.20.0 and later, measurementId is optional
  const firebaseConfig = {
    apiKey: "AIzaSyB_Qlovx15Cx_-h5TPzaBog8oRW4e8yk8",
    authDomain: "blog-930aa.firebaseio.com",
    projectId: "blog-930aa",
    storageBucket: "blog-930aa.appspot.com",
    messagingSenderId: "252280810021",
    appId: "1:252280810021:web:e6668c21a58d92edd86dcc",
    measurementId: "G-R7TG59DGRQ"
  };

  // Initialize Firebase
  const app = initializeApp(firebaseConfig);
  const analytics = getAnalytics(app);
</script>
```



Activer Cloud Firestore

- Vous pouvez activer la base de données Firestore en sélectionnant Firestore dans le menu de gauche, puis en cliquant sur Créer une base de données.



Ajout de Firebase à Flutter

- `firebase_core` : requis pour l'initialisation de Firebase et l'utilisation de tout autre plugin Firebase.
- `cloud_firestore` : requis pour interagir avec la base de données Firestore.

```
dependencies:  
  flutter:  
    sdk: flutter  
  firebase_core: ^1.11.0  
  cloud_firestore: ^3.1.6
```



Initialisation

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: const FirebaseOptions(
      apiKey: "AIzaSyB_Qlovx15Cx_-h5TPzaBog8oRW4e8ykd8",
      authDomain: "blog-930aa.firebaseio.com",
      projectId: "blog-930aa",
      storageBucket: "blog-930aa.appspot.com",
      messagingSenderId: "252280810021",
      appId: "1:252280810021:web:e6668c21a58d92edd86dcc",
      measurementId: "G-R7TG59DGRQ"
    )
  );
  runApp(const MyApp());
}
```

Model

```
class Article {
    Article({
        this.id,
        required this.title,
        required this.body,
        this.createdAt,
    });

    final String? id;
    final DateTime? createdAt;
    final String title;
    final String body;

    factory Article.fromJson(QueryDocumentSnapshot query) => Article(
        id: query.id,
        title: query["title"],
        body: query["body"],
        createdAt: (query.data() as Map).containsKey("createdAt") ? query["createdAt"].toDate() :
null,
    );

    Map<String, dynamic> toJson() => {
        "title": title,
        "body": body,
        "createdAt": DateTime.now(),
    };
}
```

Ajouter un document

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

```
final FirebaseFirestore firestore = FirebaseFirestore.instance;
```

```
Article article =Article(  
  title: "codicem nondum per aetatem firmato",  
  body: "Circa Phalangio Baeticae consulari cecidit funesti  
carnificis."  
);  
firestore.collection('articles').add(  
  article.toJson()  
);
```



Afficher liste des documents

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

```
final FirebaseFirestore firestore = FirebaseFirestore.instance;
```

```
FutureBuilder(  
  future: firestore.collection("articles")  
    .orderBy("createdAt",descending: true)  
    .get(),  
  builder: (context, snapshot) {  
    if(snapshot.hasError){  
      return onError(message: snapshot.error.toString());  
    } else if(snapshot.hasData){  
      QuerySnapshot collection = snapshot.data as QuerySnapshot;  
      return BuildListArticle(collection: collection);  
    } else {  
      return const Center(  
        child: CircularProgressIndicator()  
      );  
    }  
  },  
)
```


Afficher liste des documents

```
import 'package:cloud_firestore/cloud_firestore.dart';
```

```
final FirebaseFirestore firestore = FirebaseFirestore.instance;
```

```
StreamBuilder(  
  stream: firestore.collection("articles")  
    .orderBy("createdAt",descending: true)  
    .snapshots(includeMetadataChanges: true),  
  builder: (context, snapshot) {  
    if(snapshot.hasError){  
      return onError(message: snapshot.error.toString());  
    } else if(snapshot.hasData){  
      QuerySnapshot collection = snapshot.data as  
QuerySnapshot;  
      return BuildListArticle(collection: collection);  
    } else {  
      return const Center(  
        child: CircularProgressIndicator()  
      );  
    }  
  },  
)
```

Afficher liste des documents

```
class BuildListArticle extends StatelessWidget {
  const BuildListArticle({
    Key? key,
    required this.collection,
  }) : super(key: key);

  final QuerySnapshot<Object?> collection;

  @override
  Widget build(BuildContext context) {
    return ListView.builder(
      itemCount: collection.docs.length,
      itemBuilder: (context, index) {
        Article article = Article.fromJson(collection.docs[index]);
        return ItemCard(article: article);
      },
    );
  }
}
```

Afficher liste des documents

```
class ItemCard extends StatelessWidget {
  const ItemCard({
    Key? key,
    required this.article,
  }) : super(key: key);
  final Article article;
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(8.0),
      child: Card(
        elevation: 8,
        child: Column(
          children: [
            Align(
              alignment: Alignment.centerRight,
              child: Text(article.createdAt!.toIso8601String()),
            ),
            ListTile(
              title: Text(article.title),
              subtitle: Text(article.body),
            ),
          ],
        ),
      ),
    );
  }
}
```

Afficher liste des documents

```
class OnError extends StatelessWidget {  
  final String message;  
  const OnError({  
    Key? key,  
    required String this.message,  
  }) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Text(message,  
        style: const TextStyle(color: Colors.red),  
      ),  
    );  
  }  
}
```

Encore plus de notion :

- Lectures écritures dans un fichier
- La base de données SQL Lite avec Flutter
- Passage de paramètres entre pages
- Déploiement sur Play store et Apple store

